

Oracle Dynamic SQL

Native Dynamic SQL (EXECUTE IMMEDIATE) & DBMS_SQL · Oracle 19c

Dynamic SQL is SQL/PL/SQL built and executed at run time, when the statement text, table/column names, or structure aren't known until execution. Oracle offers two mechanisms: **Native Dynamic SQL (NDS)** via `EXECUTE IMMEDIATE` / `OPEN FOR` (preferred), and the **DBMS_SQL** package (for fully unknown column lists).

1. EXECUTE IMMEDIATE — the workhorse

```
-- DDL (cannot be run directly in PL/SQL otherwise)
EXECUTE IMMEDIATE 'CREATE TABLE tmp_x (id NUMBER)';

-- DML with bind variables
EXECUTE IMMEDIATE
  'UPDATE emp SET sal = sal * :pct WHERE dept_id = :d'
  USING 1.10, 20;
```

2. Querying: single row & multi-row

```
-- single row INTO
DECLARE l_name VARCHAR2(50);
BEGIN
  EXECUTE IMMEDIATE 'SELECT ename FROM emp WHERE id = :1'
    INTO l_name USING 7788;
END;
/

-- multi-row BULK COLLECT
DECLARE
  TYPE t IS TABLE OF emp%ROWTYPE;
  l t;
BEGIN
  EXECUTE IMMEDIATE 'SELECT * FROM emp WHERE dept_id = :d'
    BULK COLLECT INTO l USING 30;
END;
/
```

3. OPEN FOR — dynamic cursors (ref cursor)

```
DECLARE
  l_cur SYS_REFCURSOR;
  l_where VARCHAR2(200) := 'sal > :min_sal';
  l_rec emp%ROWTYPE;
BEGIN
  OPEN l_cur FOR 'SELECT * FROM emp WHERE ' || l_where
    USING 5000;
  LOOP
    FETCH l_cur INTO l_rec;
    EXIT WHEN l_cur%NOTFOUND;
    DBMS_OUTPUT.put_line(l_rec.ename);
  END LOOP;
  CLOSE l_cur;
END;
/
```

4. RETURNING & OUT binds

```
DECLARE l_new_sal NUMBER;
BEGIN
  EXECUTE IMMEDIATE
    'UPDATE emp SET sal = sal + 100 WHERE id = :1 RETURNING sal INTO :2'
    USING 7788 RETURNING INTO l_new_sal;
END;
/
```

5. Bind Variables vs Concatenation

SQL Injection risk! Never concatenate user input as *values*. Use **USING** bind variables. Concatenation is only acceptable for *identifiers* (table/column names) that you validate yourself.

Bad (injectable)	Good (bind)
'... WHERE name='' p_in '''	'... WHERE name=:1' USING p_in

Validate dynamic identifiers with **DBMS_ASSERT**:

```
EXECUTE IMMEDIATE
  'SELECT COUNT(*) FROM ' || DBMS_ASSERT.sql_object_name(p_table)
  INTO l_cnt;
```

6. Dynamic FORALL (bulk + dynamic)

```
FORALL i IN 1 .. l_id.COUNT
  EXECUTE IMMEDIATE
    'UPDATE ' || l_table_name || ' SET sal=:1 WHERE id=:2'
    USING l_sal(i), l_id(i);
```

7. DBMS_SQL — when columns are unknown at compile time

Use **DBMS_SQL** for "method 4" dynamic SQL: variable number of columns/binds not known until run time (e.g. a generic query tool).

```
DECLARE
  l_cur  INTEGER;
  l_rows INTEGER;
  l_val  VARCHAR2(100);
BEGIN
  l_cur := DBMS_SQL.open_cursor;
  DBMS_SQL.parse(l_cur, 'SELECT ename FROM emp WHERE dept_id = :d', DBMS_SQL.native);
  DBMS_SQL.bind_variable(l_cur, ':d', 20);
  DBMS_SQL.define_column(l_cur, 1, l_val, 100);
  l_rows := DBMS_SQL.execute(l_cur);
  WHILE DBMS_SQL.fetch_rows(l_cur) > 0 LOOP
    DBMS_SQL.column_value(l_cur, 1, l_val);
    DBMS_OUTPUT.put_line(l_val);
  END LOOP;
  DBMS_SQL.close_cursor(l_cur);
END;
/
```

8. NDS vs DBMS_SQL

Aspect	Native Dynamic SQL	DBMS_SQL
--------	--------------------	----------

Ease of use	Simple, concise	Verbose (many calls)
Performance	Faster (compiled once)	Slower
Unknown column list	No	Yes (describe_columns)
Recommendation	Default choice	Only when NDS can't do it

Best practices

- Always prefer **bind variables** — security (no injection) and performance (cursor sharing).
- Use **DBMS_ASSERT** to validate dynamic table/column names.
- Keep dynamic SQL to a minimum; static SQL is checked at compile time and is faster.
- For DDL inside PL/SQL, **EXECUTE IMMEDIATE** is the only option (DDL also does an implicit COMMIT).